2026학년도 2학기 언어데이터과학

제12강 모두의 말뭉치 (2) ‘완전’과 ‘아주’의 출현 환경 비교

박수민

서울대학교 인문대학 언어학과

2026년 10월 19일 월요일

지난 시간에 배운 것

1. JSON의 객체는 이름으로, 배열은 자리로 값을 찾는다.
2. 말뭉치의 파일 47,421개를 발화 표와 화자 표, 두 CSV로 펼쳤다.
3. 빈 발화 92,087개를 빼고 NFC로 맞추었다. 무엇을 빼고 고쳤는지 적어 두었다.

오늘의 목표

1. 논문의 물음을 말뭉치에서 **셀 수 있는 것**으로 바꾸어 적을 수 있다.
2. 정규표현식의 단어 경계와 괄호로 낱말과 그 뒤의 어절을 찾을 수 있다.
3. 형태소 분석 결과에서 바로 뒤 형태소를 뽑아 품사별로 비교할 수 있다.
4. 두 낱말을 비율로 비교하고, 그 수가 말하지 않는 것을 말할 수 있다.

사전 준비

터미널 입력

```
cd /workspaces/LDS2026
rm -rf ~/.cache/matplotlib
```

둘째 줄은 10강에서 설치한 나눔고딕을 matplotlib이 새로 찾게 한다(7강). 11강의 CSV가 없으면 11강 노트북부터 [Run All]로 돌린다.

노트북 12-wanjeon.ipynb의 첫 셀

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from kiwipiepy import Kiwi

kiwi = Kiwi()
sns.set_theme(style='whitegrid', font='NanumGothic')
```

1 **‘완전’은 무엇인가**
사전과 논문
물음을 수로

2 글자로 찾기

3 형태소로 찾기

4 말마다 비교하기

5 마무리

사전의 ‘완전’은 명사다

표준국어대사전

완전 명사 (주로 일부 명사 앞에 쓰여) 필요한 것이 모두 갖추어져 모자람이나 흠이 없음.
¶ 금융 시장의 완전 개방.

완전히 부사 필요한 것이 모두 갖추어져 모자람이나 흠이 없이.
¶ 그 일은 완전히 끝냈다.

그런데 우리는 이렇게 말한다

(1가) 이 과자 **완전** 맛있다. (2가) 나 떡볶이 **완전** 좋아해.
명사가 형용사와 동사를 꾸민다. 사전의 ‘완전’으로는 설명되지 않는 쓰임이다. 2026년 10월에 찾아보아도 표준국어대사전의 ‘완전(完全)’은 명사뿐이다.

출처: 김다미(2021)의 (1)·(2)·(10)·(16), https://stdict.korean.go.kr

오늘 읽는 논문

김다미 (2021), 부사적 쓰임을 보이는 ‘완전(完全)’에 대한 통시적 고찰, 《국어학》 97, 439–475

15세기부터 지금까지의 문헌과 21세기 세종 계획의 형태 분석 말뭉치(현대 문어·구어)에서 ‘완전’의 용례를 찾아 읽고, 쓰임이 어떻게 넓어졌는지를 따라간 논문이다.

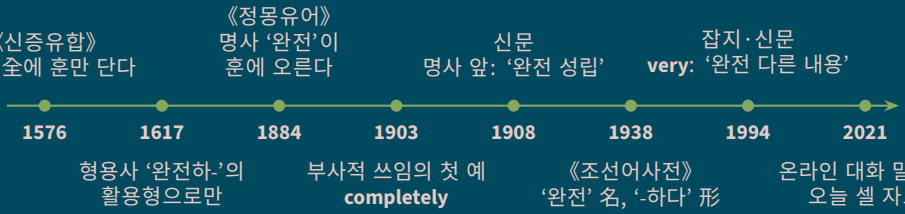
논문이 던진 물음

1. ‘완전’의 부사적 쓰임은 언제부터 나타났는가?
2. 어느 뜻이 먼저이고, 어느 뜻이 나중에 생겼는가?
3. 그 확장을 이끈 힘은 무엇인가?

‘완전’의 짝으로 비교한 말이 부사 ‘아주’다. 오늘 우리도 이 둘을 비교한다.

출처: https://doi.org/10.15811/jkl.2021.97.014

‘완전’이 걸어온 길



덜 추상적인 뜻에서 더 추상적인 뜻으로

명사 ‘완전’이 명사 앞에서 꾸미는 말로 쓰이다가, 20세기 초에 동사구 앞의 completely가, 20세기 말에야 형용사 앞의 very가 나타났다. 논문은 very로 활발히 쓰이게 된 것을 1990년대 중후반 또는 2000년대 초부터의 일로 본다(464쪽).

두 가지 ‘완전’

	완전[1]	완전[2]
뜻	모든 측면에서 두루, 완벽하게 (**completely**)	보통 정도보다 훨씬 더 (**very**)
어울리는 말	동작성 서술어 — **동사**	상태성 서술어 — **형용사**
예	그들은 완전 남이 되었다. 공이 완전 펜스를 넘겼다.	나 떡볶이 완전 좋아해. 철수는 고수를 완전 잘 먹는다.
‘완전히’로	바꿀 수 있다 노트북이 **완전히** 망가졌다.	어색하다 ??기분이 **완전히** 좋다.

부사 ‘완전히’는 완전[1]의 자리만 맡는다 김다미(2021)의 (2), (17), (18)

같은 뜻의 두 낱말은 흔히 하나가 밀려나는데, ‘완전’과 ‘완전히’는 뜻을 나누어 함께 산다.
‘좋아하다’는 동사지만 ‘완전 좋아해’는 very다. 품사와 뜻은 **경향**으로 이어질 뿐이다.

‘아주’도 둘로 나뉜다

표준국어대사전의 ‘아주’

1. (형용사 또는 상태의 뜻을 나타내는 일부 동사나 명사, 부사 앞에 쓰여) 보통 정도보다 훨씬 더 넘어선 상태로. ¶ 이번 시험 문제는 아주 쉽다. ¶ 아주 가끔씩 그는 나에게 …
2. (동사 또는 일부의 명사적인 성분 앞에 쓰여) 어떤 행동이나 작용 또는 상태가 이미 완전히 이루어져 달리 변경하거나 더 이상 어찌할 수 없는 상태에 있음을 나타내는 말. ¶ 두 사람은 아주 남남이 되어 버렸다.

	completely (동사 앞)	very (형용사 앞)
아주(아조)	16–18세기	19세기 말 (1892, ‘아조 쉬울지라’)
완전	20세기 초 (1903)	20세기 말 (1994, ‘완전 다른 내용’)

같은 길을 ‘아주’가 먼저 갔다. 논문은 ‘아주’가 very를 기본 뜻으로 삼을 만큼 나아간 반면 ‘완전’은 그 속도에 미치지 못했다고 본다(462–463쪽).

논문의 가설과 오늘의 물음

논문이 세운 가설 (4.2절)

‘완전개방하-’ 같은 말이 ‘완전 + 개방하-’로 다시 분석되면서 ‘완전’이 부사로 쓰이는 일이 실제보다 잦게 느껴졌다. 게다가 ‘완전’은 ‘완전히’보다 음절이 적고 접사가 없어 경제적이다. 그래서 ‘완전히’보다 **더 자주** 쓰였고, 자주 쓰이면서 뜻이 더 넓어졌다.

2021년 온라인 대화 3백만 발화에서 셀 것

1. ‘완전’은 ‘완전히’, ‘아주’와 비교해 얼마나 자주 쓰이는가?
2. ‘완전’과 ‘아주’의 **바로 뒤**에는 어떤 품사가 오는가?
3. 어떤 말이 ‘완전’을, 어떤 말이 ‘아주’를 더 좋아하는가?

논문은 용례를 읽어 뜻을 갈랐다. 오늘은 같은 두 낱말을 말뭉치에서 센다.

물음을 수로 바꾸기

논문의 말	오늘 셀 것
부사적 쓰임의 ‘완전’	띄어 쓴 어절 ‘완전’(§2), 형태소 ‘완전’(§3)
상태성 서술어 (완전[2])	바로 뒤 형태소가 **형용사** (VA, 어근 XR)
동작성 서술어 (완전[1])	바로 뒤 형태소가 **동사** (VV)
명사 앞에서 꾸미는 말	바로 뒤 형태소가 **체언** (NNG, NNP, NP)

조작적 정의(operational definition)

개념을 **무엇을 어떻게 세는가**로 바꾸어 적은 것이다. 8강의 “무엇을 하나로 셀 것인가”를 논문의 개념에 대고 묻는 일이다. 다른 사람이 같은 수를 다시 얻으려면 이것을 적어야 한다.

품사는 뜻의 **어림값**이다. ‘완전 좋아해’처럼 어림이 빗나가는 자리도 함께 본다.

1 ‘완전’은 무엇인가

2 **글자로 찾기**
단어 경계
바로 뒤 어절

3 형태소로 찾기

4 말마다 비교하기

5 마무리

발화 표를 다시 읽는다

노트북 입력

```
u = pd.read_csv('data/nikl/utterances.csv')
print(u.shape)
```

실행 결과

```
(2977840, 6)
```

11강에서 남긴 그대로

빈 발화를 빼고 NFC로 맞춘 2,977,840행이다. 오늘 쓰는 열은 발화를 다듬은 `form` 하나다. 이름이 길어 오늘은 표를 u라고 부른다.

‘아주’가 든 발화

노트북 입력

```
has = u['form'].str.contains('아주')
words = u.loc[has, 'form'].str.findall(r'\w*아주\w*').explode()
print(words.value_counts().head(6))
```

실행 결과

```
form
아주       3397
아주그냥     83
아주아주     46
놀아주고     40
아주스틸     33
아주머니     31
Name: count, dtype: int64
```

`\w` 글자 하나. 한글, 로마자, 숫자… 문장 부호와 공백은 아니다

`findall` 맞는 부분을 **모두** 목록으로 돌려준다(8강의 `re.findall`)

`\w*아주\w*`는 ‘아주’를 품은 낱말 전체다. ‘놀아주고’(-아 주다), ‘아주머니’, 주식 종목 ‘아주스틸’도 걸렸다. **글자열 ‘아주’는 낱말 ‘아주’가 아니다.**

단어 경계 \b

노트북 입력

```
for w in ['완전', '완전히', '아주']:
    print(w, u['form'].str.count(rf'\b{w}\b').sum())
```

실행 결과

```
완전 8636
완전히 174
아주 3397
```

첫째 물음의 답

‘완전’은 ‘완전히’의 약 50배, ‘아주’의 2.5배다. 논문의 가설이 기대한 쪽과 같다.

\b 글자(\w)와 글자가 아닌 것 사이. 낱말의 앞뒤 끝이다. ‘완전히’의 ‘완전’ 뒤에는 ‘히’가 붙어 있어 경계가 아니다

rf'' r은 백슬래시를 그대로 두고(8강), f는 {w} 자리에 값을 넣는다(10강)

str.count 발화마다 맞은 **횟수**. 더하면 출현 수다

\b가 놓친 '완전'

노트북 입력

```
words = u['form'].str.findall(r'\w*완전\w*').explode()
print(words[words != '완전'].value_counts().head(6))
```

실행 결과

```
form
완전히      174
완전요       70
완전한       32
완전좋아      20
완전좋아해요     18
이거완전       14
Name: count, dtype: int64
```

놓친 것 완전요, 완전좋아, 완전좋아해요, 이거완전

바르게 뺀 것 완전히, 완전한 — 다른 낱말이다

붙여 쓴 '완전좋아'와 다른 낱말 '완전한'은 둘 다 '완전' 뒤에 한글이 붙었다. **글자로는 가를 수 없다.** 어디서 끊어지는지는 형태소 분석이 알려 준다(§3).

괄호로 뒤의 어절을 잡는다

노트북 입력

```
wj = u['form'].str.findall(r'\b완전 (\w+)').explode().value_counts()
aj = u['form'].str.findall(r'\b아주 (\w+)').explode().value_counts()
ej = pd.DataFrame({'완전': wj, '아주': aj}).fillna(0).astype(int)
print(ej.sort_values('완전', ascending=False).head(4))
```

실행 결과

```
          완전  아주
form
좋아해요  173   13
많이       58   23
잘         50   59
맛있어요   38    7
```

( ) 괄호(묶음) 안에 맞은 부분만 돌려준다. 뒤의 어절만 남는다

fillna(0) 한쪽에만 나온 어절은 다른 쪽이 NaN이다. 0으로 메운다(7강)

DataFrame에 '열 이름: Series'의 dict를 주면 낱말이 같은 행끼리 맞추어 붙인다.

어절로 세면 '좋다'가 흩어진다

노트북 입력

```
good = ej[ej.index.str.startswith('좋')]
print(len(good))
print(good.sort_values('완전', ascending=False).head(5))
```

실행 결과

```
242
          완전  아주
form
좋아해요  173   13
좋아요     34   22
좋아       26   25
좋죠       24    7
좋은       21   52
```

10강의 '영화' 6,233가지와 같은 일

'좋'으로 시작하는 어절만 242가지다. 게다가 '좋아해요'는 동사 '좋아하다', '좋은'은 형용사 '좋다'다. 어절은 어미까지 달고 있어서 **품사로 묶으려면 형태소로 나누어야** 한다.

확인 질문

글자로 찾기

1. `str.contains('아주')`로 고른 발화에는 무엇이 섞여 드는가? 두 가지를 들어 보자.
2. `r'\b완전\b'`은 '완전히'에서 맞지 않는다. '완전!!'에서는 맞을까? '완전ㅋㅋ'에서는?
3. `r'\b완전 (\w+)'`에서 괄호를 빼면 `findall`은 무엇을 돌려줄까?
4. '완전'이 '아주'보다 2.5배 많다는 것만으로 '완전'이 '아주'를 밀어냈다고 말할 수 있을까?

1 ‘완전’은 무엇인가

2 글자로 찾기

3 **형태소로 찾기**
- Kiwi가 본 ‘완전’
- 바로 뒤 형태소
- 품사로 묶기

4 말마다 비교하기

5 마무리

걸러 놓고 나눈다

노트북 입력

```
def pos(tokens):
    return [(t.form, t.tag) for t in tokens]

sub = u[u['form'].str.contains('완전|아주')]
sub['morphs'] = [pos(ts) for ts in kiwi.tokenize(sub['form'])]
print(len(sub))
```

실행 결과

```
16131
```

3백만 발화를 다 나누지 않는다. 글자 '완전'이나 '아주'가 든 16,131발화만 걸러 Kiwi에 넘긴다. |는 '또는'이다(3강). pos는 10강의 그 함수다. '놀아주고'도 들어오지만 괜찮다. **거를 때는 넉넉하게, 셀 때는 엄격하게.**

Kiwi가 붙인 품사

노트북 입력

```
m = sub['morphs'].explode()
mm = pd.DataFrame(m.tolist(), columns=['form', 'tag'])
print(mm[mm['form'].isin(['완전', '완전히', '아주'])].value_counts())
```

실행 결과

```
form  tag
완전    MAG     9177
아주    MAG     4234
완전    NNG      783
완전히   MAG      181
아주    NNP        5
Name: count, dtype: int64
```

Kiwi는 '완전'을 대개 부사로 보았다

9,177번은 일반 부사(MAG), 783번은 명사(NNG)다. 사전과 다르다. 표의 `value_counts`는 (형태, 태그) **짝**을 센다.

형태소로 찾은 '완전' 9,960은 `\b`로 찾은 8,636보다 많다. '완전좋아', 'ㅋㅋㅋ완전'처럼 붙여 쓴 것까지 찾았기 때문이다.

논문의 예문을 Kiwi에 넣으면

노트북 입력 — (1가), (1라), (2다), (2라)

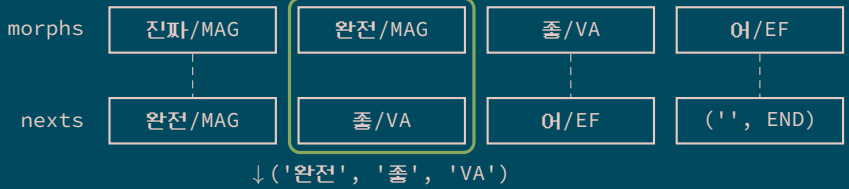
```
for s in ['이 과자 완전 맛있다.', '오늘 시험 완전 어렵다.',
          '그들은 완전 남이 되었다.', '공이 완전 펜스를 넘겼다.']:
    print(' '.join(f'{t.form}/{t.tag}' for t in kiwi.tokenize(s)))
```

실행 결과

```
이/MM 과자/NNG 완전/MAG 맛있/VA 다/EF ./SF
오늘/MAG 시험/NNG 완전/NNG 어렵/VA-I 다/EF ./SF
그/NP 들/XSN 은/JX 완전/NNG 남/NNG 이/JKC 되/VV 었/EP 다/EF ./SF
공/NNG 이/JKS 완전/NNG 펜스/NNG 를/JKO 넘기/VV 었/EP 다/EF ./SF
```

넷 다 부사적 쓰임인데 셋이 NNG다. 분석기도 흔들린다. 그래서 오늘은 **태그와 상관없이 형태 '완전'을 모두** 센다.

한 칸 당겨 짝짓기



‘진짜 완전 좋아’의 형태소 넷

형태소 목록 `morphs`를 한 칸 당긴 목록 `nexts`를 만들어 위아래로 짝지으면, 자리마다 **바로 뒤 형태소**가 아래에 온다. 마지막 형태소 아래에는 ‘끝’을 뜻하는 `END`를 채운다. 위가 ‘완전’이나 ‘아주’인 짝만 고른다.

이웃한 두 형태소의 짝(bigram)은 20강의 분포 가설에서 다시 만난다.

after — 바로 뒤 형태소를 고르는 함수

노트북 입력

```
def after(morphs):
    nexts = morphs[1:] + [('', 'END')]
    found = []
    for (w, _), (f, t) in zip(morphs, nexts):
        if w in ('완전', '아주'):
            found.append((w, f, t))
    return found

print(after(pos(kiwi.tokenize('진짜 완전 좋아'))))
```

실행 결과

```
[('완전', '좋', 'VA')]
```

morphs[1:] 한 칸 당긴 목록

zip 두 목록을 자리마다 짝짓는다

_ 쓰지 않을 값(태그)을 받는 자리

출현 한 번이 한 행

노트북 입력

```
rows = []
for morphs in sub['morphs']:
    rows += after(morphs)
a = pd.DataFrame(rows, columns=['word', 'next', 'tag'])
a['lemma'] = a['next'] + '/' + a['tag']
print(a['word'].value_counts())
```

실행 결과

```
word
완전    9960
아주    4239
Name: count, dtype: int64
```

무엇을 하나로 세는가 (8강)

행 하나는 발화가 아니라 '완전'이나 '아주'의 **출현 한 번**이다. +=는 돌려받은 목록을 뒤에 이어 붙인다. lemma는 '좋/VA'처럼 형태와 태그를 이은 열이다.

맨 위를 눈으로 읽는다

노트북 입력

```
print(a[a['word'] == '아주']['lemma'].value_counts().head(8))
```

실행 결과

```
lemma
좋/VA       299
/END        278
잘/MAG      162
그냥/MAG    162
가끔/MAG    124
아주/MAG     71
좋아하/VV    60
스틸/NNG     49
Name: count, dtype: int64
```

읽는 법

/END는 '아주'로 발화가 끝난 것이다(형태는 빈 문자열). '아주 좋다', '아주 잘', '아주 그냥', '아주 가끔', '아주아주'까지는 짐작대로다.

그런데 '아주 스틸'?

49번이나 나왔다. 무슨 말일까?

‘아주스틸’은 주식 종목이다

노트북 입력

```
print(sub.loc[sub['form'].str.contains('아주스틸'), 'form'].head(3))
a = a[a['next'] != '스틸']
print(len(a))
```

실행 결과

```
240556    월요일 청약 \r\n미래에셋 단독 아주스틸 \r\n그리고 롯데렌탈
241135                          반면에 아주스틸은 오를거같대
241137                             응 아주스틸은 좋아보여
Name: form, dtype: str
14150
```

공모주 이야기다. 분석기는 처음 보는 고유명사를 아는 말로 쪼갠다. ‘완전 스틸’은 없으니 ‘아주’ 쪽 49행만 빠진다. **빼고, 뺐다고 적어 둔다**(10강, 11강).

태그를 묶음으로

노트북 입력

```
groups = {'VA': '형용사', 'VA-I': '형용사', 'VA-R': '형용사',
          'XR': '형용사', 'MAG': '부사', 'END': '끝',
          'VV': '동사', 'VV-I': '동사', 'VV-R': '동사',
          'NNG': '체언', 'NNP': '체언', 'NP': '체언'}
a['group'] = a['tag'].map(groups).fillna('기타')
print(a['group'].value_counts().to_dict())
```

실행 결과

```
{'체언': 5585, '형용사': 3734, '부사': 1775, '동사': 1337, '기타': 1074, '끝': 645}
```

map은 사전에 있는 태그를 묶음 이름으로 바꾸고, 없는 태그는 NaN으로 둔다. fillna가 그것을 '기타'로 메운다(문장 부호, 'ㅋ', 관형사 등). 어근 XR은 '멀쩡', '궁금'처럼 '-하다'가 붙어 형용사가 된다. **어떻게 묶었는지도 조작적 정의의 일부**다.

비율로 비교한다

노트북 입력

```
order = ['형용사', '동사', '체언', '부사', '끝', '기타']
pct = pd.crosstab(a['group'], a['word'], normalize='columns') * 100
print(pct.loc[order].round(1))
```

실행 결과

word	아주	완전
group		
형용사	29.4	25.1
동사	5.0	11.3
체언	29.1	43.8
부사	22.8	8.2
끝	6.6	3.7
기타	7.1	7.8

`normalize` `'columns'`이면 열마다 합이 1이 되게 나눈다. 100을 곱하면 백분율이다

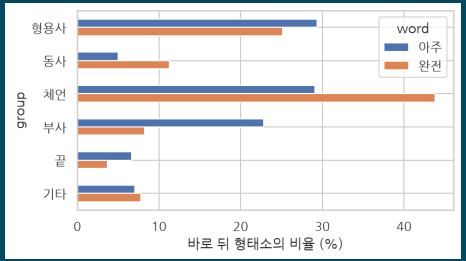
`loc[order]` 행을 정한 순서로 늘어놓는다

‘완전’(9,960)은 ‘아주’(4,190)보다 2.4배 많다. 개수를 그대로 놓으면 어느 줄에서나 ‘완전’이 크다. **모두가 다른 둘은 비율로 비교한다.**

막대그림으로

노트북 입력

```
ax = pct.loc[order].plot.barh(figsize=(6, 3.2))
ax.invert_yaxis()
ax.set_xlabel('바로 뒤 형태소의 비율 (%)')
plt.savefig('plots/wanjeon-aju.png', dpi=150, bbox_inches='tight')
```



표에도 그리기가 붙어 있다. plot.barh는 행마다 가로 막대를, 열마다 색을 하나씩 쓴다. 가로 막대는 아래부터 쌓으므로 invert_yaxis로 뒤집어 표와 같은 순서로 둔다. plots/는 7강에서 만들었다.

둘째 물음의 답

형용사 둘 다 4분의 1 남짓(29.4, 25.1)이다. ‘완전’도 very로 넉넉히 쓰인다
동사 ‘완전’ 11.3, ‘아주’ 5.0. 동사 앞에는 ‘완전’이 두 배 넘게 자주 온다
체언 ‘완전’의 43.8%가 명사 앞이다. ‘완전 가을이야’, ‘완전 공감’, ‘완전 대박’
부사 ‘아주’의 22.8%가 부사 앞이다. ‘아주 잘’, ‘아주 그냥’, ‘아주 가끔’

두 부사는 형용사 앞에서 만나고, 명사 앞과 부사 앞에서 갈린다

‘완전’이 명사 앞에 많은 것은 명사 ‘완전’의 오랜 자리(완전 개방)와 이어져 보인다. 다만 ‘완전 가을이야’의 ‘완전’은 명사 ‘가을’이 아니라 ‘가을이다’ 전체를 꾸민다. 바로 뒤 하나만 보는 자가 놓치는 자리다.

‘끝’을 조심해 읽는다

채팅에서는 한 문장을 여러 메시지로 끊어 보낸다(‘와우’ / ‘완전’ / ‘꽁스네’). 오늘은 한 발화 안에서만 바로 뒤를 찾았으므로 이런 짝은 ‘끝’이 된다.

1 ‘완전’은 무엇인가

2 글자로 찾기

3 형태소로 찾기

4 **말마다 비교하기**
무엇 앞에 오나
저장소에 남기기

5 마무리

형용사 앞과 동사 앞

노트북 입력

```
for g in ['형용사', '동사']:
    for w in ['완전', '아주']:
        x = a[(a['group'] == g) & (a['word'] == w)]
        print(g, w, list(x['lemma'].value_counts().index[:6]))
```

실행 결과

```
형용사 완전 ['좋/VA', '맛있/VA', '다르/VA', '재밌/VA', '귀엽/VA-I', '부럽/VA-I']
형용사 아주 ['좋/VA', '맛있/VA', '어리/VA', '많/VA', '멋지/VA', '귀엽/VA-I']
동사 완전 ['좋아하/VV', '웃기/VV', '빠지/VV', '맞/VV', '신나/VV', '미치/VV']
동사 아주 ['좋아하/VV', '신나/VV', '미치/VV', '죽/VV', '웃기/VV', '가/VV']
```

동사 앞 '완전'의 1·2위 '좋아하다'(390번)·'웃기다'(112번)는 정도를 말하니 very, 형용사 앞 '완전 다르다'(161번)는 completely다. **품사는 뜻의 어림값이다.**

어느 쪽을 더 좋아하나 — '아주' 쪽

노트북 입력 — 내용어 앞만 남기고, 모든 칸에 1을 더해 나눈다

```
c = a[a['group'].isin(['형용사', '동사', '체언', '부사'])]
ct = pd.crosstab(c['lemma'], c['word']) + 1
ct['완전/아주'] = ct['완전'] / ct['아주']
print(ct.sort_values('완전/아주').head(5))
```

실행 결과

word	아주	완전	완전/아주
lemma			
아주/MAG	72	1	0.013889
가끔/MAG	125	5	0.040000
그냥/MAG	163	7	0.042945
잠깐/MAG	17	1	0.058824
조금/NNG	25	2	0.080000

비가 작을수록 '아주' 쪽

'아주 아주'(겹쳐 쓰기) 다음은 가끔, 그냥, 잠깐, 조금이다. 살짝(23 대 1), 예전(37 대 10), 어리-(43 대 25), 작-(23 대 12)도 '아주' 쪽이다. **작은 정도, 짧은 때, 먼 옛날**이다.

어느 쪽을 더 좋아하나 — '완전' 쪽

노트북 입력 — 정렬한 표의 맨 끝

```
print(ct.sort_values('완전/아주').tail(5))
```

실행 결과

word	아주	완전	완전/아주
lemma			
추억/NNG	1	35	35.0
가을/NNG	2	72	36.0
똑같/VA	1	36	36.0
공감/NNG	2	73	36.5
싫/VA	1	42	42.0

비가 클수록 '완전' 쪽

싫-, 공감, 똑같-, 가을, 추억. 그 아래로 반대, 신기하-, 추천, 다르-(6 대 161), 대박, 웃기-가 보인다. **같음과 다름, 그리고 명사 서술어** ('완전 가을이야', '완전 공감')다.

ct의 수는 1을 더한 것이다. '싫다'는 실제로 '아주' 0번, '완전' 41번이다.

1을 더한 까닭

0으로 나눌 수는 없다

‘싫다’(‘아주’ 0번, ‘완전’ 41번)처럼 한쪽이 0인 말은 비가 0이나 무한대가 된다. 그러면 한 번 나온 말과 백 번 나온 말이 같은 자리에 선다. 모든 칸에 1을 더하면(**가법 평활**, add-one smoothing) 0이 사라지고, 한두 번 나온 말은 비가 1 가까이로 끌려와 **여러 번 나온 말만** 양 끝에 남는다.

	센 그대로		1을 더하면	
	아주 / 완전	비	아주 / 완전	비
싫다	0 / 41	무한대	1 / 42	42.0
가끔	124 / 4	0.032	125 / 5	0.040
어떤 한 번 나온 말	0 / 1	무한대	1 / 2	2.0

더하는 수(1)도 연구자가 정한 것이다. 정한 것은 적어 둔다.

셋째 물음의 답

	더 좋아하는 말	묶어 보면
‘아주’ 쪽	가끔, 그냥, 잠깐, 조금, 살짝, 예전, 어리-, 작-	작은 정도, 짧은 때, 먼 옛날
‘완전’ 쪽	싫-, 공감, 똑같-, 가을, 추억, 반대, 다르-	같음과 다름, 명사 서술어

논문과 비교해 읽으면

‘아주 조금’, ‘아주 가끔’은 자연스럽고, 사전도 ‘아주’의 예문으로 ‘아주 가끔씩’을 든다. ‘아주’는 정도만 남은 very가 되어 작은 쪽의 정도까지 키운다. ‘완전’은 very로 널리 쓰이지만 아직 ‘모두, 두루’(completely)가 남아, ‘똑같다’, ‘다르다’와 잘 어울리고 작은 정도와는 잘 어울리지 않는다. 논문이 말한 **두 낱말의 속도 차이**와 같은 쪽을 가리킨다.

이것은 수를 읽는 **한 가지 방법**이다. 수 자체가 뜻을 알려 주지는 않는다.

드물다와 없다

노트북 입력 — 표의 수 뒤에 있는 발화를 읽는다

```
for f in sub.loc[sub['form'].str.contains('완전 가끔'), 'form']:
    print(f)
```

실행 결과

```
그래서 아빠는 완전 가끔 집에 오잖아
컴퓨터로 하는건 너무 어지러워서 못하고 핸드폰도 완전 가끔가끔해요 친구들이 그때그때 하자는거... 한
↪  10년전에는 닌텐도 엄청 열심히했죠
으학ㅋㅋㅋ완전 가끔이에요
완전 가끔 회식때는 한잔씩 먹어욬ㅋㅋ 한모금 마시고
```

‘완전 가끔’도 4번 있다. ‘아주 가끔’ 124번과 비교하면 드물 뿐이다. 말뭉치는 ‘안 된다’를 말하지 않는다. 수는 **경향**을 보여 줄 뿐이고, 문법성은 용례를 읽고 직관과 함께 판단한다.

이 수가 말하지 않는 것

누가 썼나 한 사람이 '완전'을 백 번 쓰면 백 번이다. 사람마다 나누어 보지 않았다

누구의 말뭉치 화자 168,267명 가운데 여성이 87.5%, 20대가 46.5%다(11강의 화자 표)

분석기의 오류 '아주스틸'은 뺐지만, '조아'(좋아)를 고유 명사로 본 것 같은 오류는 남았다

바로 뒤 하나 '완전 내 스타일'의 바로 뒤는 대명사 '내'다. 꾸밈을 받는 말은 '스타일'이다

문맥 메시지를 끊어 보낸 짝, 앞의 말('진짜 완전')은 보지 않았다

그래서 보고서에는

말뭉치와 판본, 분석기와 판본, 찾은 방법(\b, 형태소), 뺀 것('아주스틸' 49행), 묶은 방법(품사 묶음), 더한 수(1)를 적는다. 그래야 다른 사람이 같은 수를 다시 얻는다.

무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
cd /workspaces/LDS2026
git status
```

실행 결과

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        12-wanjeon.ipynb
        plots/wanjeon-aju.png

nothing added to commit but untracked files present (use "git add" to track)
```

커밋하기

터미널 입력

```
git add 12-wanjeon.ipynb plots/
git commit -m "12강 실습 노트북과 막대그림"
```

실행 결과

```
[main a1b2c3d] 12강 실습 노트북과 막대그림
 2 files changed, 739 insertions(+)
 create mode 100644 12-wanjeon.ipynb
 create mode 100644 plots/wanjeon-aju.png
```

739줄이 모두 노트북이다. 노트북 안의 막대그림은 글자로 바뀌어 한 줄로 들어 있고, PNG는 세지 않는다. 11강의 CSV는 `data/` 아래라 올라가지 않는다.

푸시하기

터미널 입력

```
git push
```

실행 결과

```
Enumerating objects: 7, done.
Writing objects: 100% (5/5), 40.54 KiB | 40.54 MiB/s, done.
To https://github.com/your-username/LDS2026.git
   e4f5a6b..a1b2c3d  main -> main
```

셈의 순서가 곧 방법이다

거르기, 나누기, 짝짓기, 빼기, 묶기, 나누어 비교하기. 오늘 정한 것이 모두 노트북에 코드로 남았다. 말뭉치를 받은 사람은 누구나 [Run All]로 같은 표와 그림을 얻는다.

1 ‘완전’은 무엇인가

2 글자로 찾기

3 형태소로 찾기

4 말마다 비교하기

5 **마무리**
- 요약
- 다음 시간
- 과제

오늘 쓴 것

하는 일	오늘의 코드
낱말을 품은 어절	`str.findall(r'\w*아주\w*')`
낱말 그 자체를 세기	`str.count(rf'\b{w}\b').sum()`
바로 뒤 어절	`str.findall(r'\b완전 (\w+)')`
빈도표 둘을 한 표로	`pd.DataFrame({'완전': wj, '아주': aj}).fillna(0)`
걸러 놓고 나누기	`kiwi.tokenize(sub['form'])`
바로 뒤 형태소	`zip(morphs, morphs[1:] + [('', 'END')])`
태그를 묶음으로	`a['tag'].map(groups).fillna('기타')`
열마다 비율	`pd.crosstab(..., normalize='columns')`
표를 그대로 그리기	`pct.plot.barh()`
평활한 비	`(ct + 1),ct['완전'] / ct['아주']`

새로 배운 것은 단어 경계, 괄호, `zip`, `map`, 평활이다. 나머지는 7-11강의 도구 그대로다.

오늘 배운 것

논문

- 완전[1] completely, 완전[2] very
- '아주'도 같은 길을 먼저 갔다
- 물음은 조작적 정의로 센다

출현 환경

- 형용사 앞에서는 둘이 비슷하다
- 명사 앞은 '완전', 부사 앞은 '아주'
- '아주'는 작은 정도까지 꾸민다

찾기

- 글자열 '아주'는 낱말이 아니다
- \b는 띄어쓰기에 기댄다
- 형태소로 찾으면 붙여 쓴 것까지

수를 읽을 때

- 모두가 다르면 비율로 비교한다
- 맨 위를 눈으로 읽는다('아주스틸')
- 드물다는 없다가 아니다

확인 질문

오늘 것을 정리해 봅시다

1. 완전[1]과 완전[2]를 영어 낱말로 옮기고, 각각 어떤 서술어와 잘 어울리는지 말해 보자.
2. 형태소로 찾은 '완전'(9,960)이 `\b`로 찾은 '완전'(8,636)보다 많은 까닭은?
3. 품사 교차표를 개수가 아니라 비율(`normalize='columns'`)로 본 까닭은?
4. '완전' 뒤에 오는 말의 TTR과 '아주' 뒤에 오는 말의 TTR을 그대로 비교하면 안 되는 까닭을 8강의 말로 설명해 보자.
5. '완전 가끔'이 4번 나왔다는 것은 오늘의 결론과 어긋나는가?

다음 시간에 배울 것 (10월 21일 수요일)

제13강 모두의 말뭉치 (3) '완전'과 '아주'의 연령별·성별 용례 비교

오늘은 '무엇 앞에' 오는지를 셌다. 다음 시간에는 '누가' 쓰는지를 센다. 11강의 `speakers.csv`를 `doc_id`와 `speaker_id` 두 열로 발화 표에 이어 붙이고, 나이와 성별로 나누어 '완전'과 '아주'의 비율을 비교한다.

미리 생각해 둘 것

10대와 40대 이상은 '완전'과 '아주'를 같은 비율로 쓸까? 화자의 87.5%가 여성인 말뭉치에서 성별을 비교할 때 무엇을 조심해야 할까?

오늘 만든 `sub`와 `a`는 노트북을 닫으면 사라진다. 13강도 `utterances.csv`를 읽는 데서 시작한다.

과제 안내

[숙제08] 언어데이터 관리 사례 조사 [발표]

마감 **2026-10-23 12:00 (금요일 정오)**. 11강에서 안내한 과제다. 부교재 *The Open Handbook of Linguistic Data Management*에서 맡은 장을 요약한 발표 자료를 `00-presentation.pdf`로 eTL에 낸다(`00`은 맡은 장의 번호).

- 발표 자료에 **퀴즈 문제 하나**를 넣고, 그 정답은 eTL의 입력란에 직접 쓴다
- 연장 신청과 질문은 **10월 22일 목요일 12:00**까지 받는다

10월 26일과 28일은 이 자료로 하는 언어데이터 관리 사례 발표다.